

UNIVERSITE DE CAEN NORMANDIE
UFR DES SCIENCES

LICENCE 3 INFORMATIQUE

AIDE À LA DÉCISION ET INTÉLLIGENCE ARTIFICIELLE

BlocksWords

Auteur :

DIALLO MARIATOU
COTCHO DEWANOU

Chargés du cours :

ZANUTUNU BRUNO
BONNET GREGORY
OUALI ABDEK KADER
CREMILLEUX BRUNO

Encadrant TP :

BEKKOUCHA DJAWAD

19 novembre 2024

Table des matières

Introduction	3
1 Classes du package blocksword	4
1.1 BWVariablesBuilder	4
1.1.1 Objectif	4
1.1.2 Fonctionnement interne	4
1.1.3 Calcul du domaine des variables	4
1.2 BWBasicConstraintsBuilder	5
1.2.1 Objectif	5
1.2.2 Méthodes principales	5
1.3 BWRegularConstraintsBuilder	6
1.3.1 Objectif	6
1.3.2 Méthodes principales	6
1.4 BWCroissanceConstraintsBuilder	7
1.4.1 Objectif	7
1.4.2 Fonctionnement	7
1.5 BWActionsBuilder	8
1.5.1 Objectif	8
1.5.2 Création des Actions	8
1.6 Classe DataminingVariableBuilder	9
1.6.1 Objectif	9
1.6.2 Structure des Variables	9
1.6.3 Création des Variables	9
1.6.4 Obtention de l'Instance de l'État	10
1.7 Les différents types d'heuristiques	11
1.7.1 Heuristique des Blocs Mal Placés	11
1.8 Heuristique de Similarité Cosinus	11
1.8.1 Explication de l'Heuristique	11
1.8.2 Calcul du Produit Scalaire et de la Norme	11
1.8.3 Estimation de la Similarité Cosinus	12
1.8.4 Exemple d'Application	12
2 Expérimentation	13
2.1 Compilation et exécution	13
2.1.1 Compilation	13
2.1.2 Exécution des démos	13
2.2 Démonstration	14
2.2.1 Vérification de satisfaction de contraintes	14
2.2.2 Création de monde	14
2.2.3 Planning	15

2.2.4	Datamining	16
	Conclusion	19

Introduction

Ce projet s'inscrit dans le cadre du cours d'aide à la prise de décision et intelligence artificielle. L'objectif principal est de développer des classes permettant de représenter des variables du monde des blocs, des contraintes pour configurer un état du monde des blocs, des actions permettant de passer d'un état du monde à un autre.

Pour y arrivé, nous avons tiré parti des concepts de satisfaction de contraintes et de planification étudiés en cours. Certaines des classes de base nécessaires à cette modélisation ont été développées en collaboration avec les professeurs lors de travaux en salle.

Il s'agit donc de partir de ces fondations pour résoudre le problème spécifique du monde des blocs, un problème classique en intelligence artificielle. Ce dernier consiste à manipuler des blocs pour les organiser selon une configuration cible, en respectant des contraintes de mouvement et d'organisation.

Ce rapport présentera dans un premier temps une description des classes développées en dehors de celles vu en cours. Enfin, nous présenterons les résultats obtenus.

Chapitre 1

Classes du package blocksworld

1.1 BWVariablesBuilder

La classe **BWVariablesBuilder** est chargée de la création des variables utilisées pour modéliser l'état du monde des blocs. Elle génère les variables associées à chaque bloc et pile en fonction du nombre de blocs et de piles spécifié lors de l'initialisation du modèle.

1.1.1 Objectif

L'objectif principal de **BWVariablesBuilder** est de créer un ensemble de variables qui représentent l'état du monde des blocs. Chaque bloc dispose de deux variables associées :

- Une variable **on** représentant la position du bloc (sur un autre bloc ou sur une pile).
- Une variable **fixed** indiquant si le bloc est fixé en place (impossible à déplacer).

Les piles sont représentées par une seule variable booléenne **free**, qui indique si la pile est libre ou occupée.

1.1.2 Fonctionnement interne

Lors de la construction d'un objet **BWVariablesBuilder**, les variables sont créées pour les blocs et les piles. Pour chaque pile, une variable (de type *BooleanVariable*) **freePile** est créée. Le nom de cette variable est un entier négatif (-1 représente donc la pile 1, -2 la pile 2 et ainsi de suite). Celle-ci, dans la suite nous permet de savoir si une pile est libre ou occupée. Pour chaque bloc, deux types de variables (de nom un entier positif) sont créés :

- Une variable **on**, dont le domaine représente les positions possibles du bloc (sur d'autres blocs ou sur une pile).
- Une variable **fixed**, de type *BooleanVariable* indiquant si le bloc est fixé ou non.

1.1.3 Calcul du domaine des variables

Les variables de type **on** ont un domaine qui représente les positions possibles où un bloc peut être placé. Le domaine de chaque bloc est calculé par la méthode **calculDomain()**. Ce domaine comprend :

- Les identifiants négatifs des piles (indiquant que le bloc peut être placé sur une pile).
- Les identifiants des autres blocs (indiquant que le bloc peut être posé sur un autre bloc, sauf sur lui-même).

1.2 BWBasicConstraintsBuilder

La classe **BWBasicConstraintsBuilder** est chargée de la définition des contraintes de base du monde des Blocks World. Elle crée des contraintes qui assurent que les blocs et les piles respectent certaines règles fondamentales, telles que :

- **Contrainte de différence entre les blocs** : Aucun bloc ne peut occuper la même position qu'un autre bloc. Autrement dit, deux blocs ne peuvent pas être placés à la même position.
- **Contrainte de fixation d'un bloc** : Si un bloc **a** est placé sur un autre bloc **b**, alors le bloc **b** doit être fixé. Cette contrainte garantit que les blocs ne peuvent pas être déplacés si un autre bloc est placé dessus.
- **Contrainte de pile libre** : Si un bloc est placé sur une pile, alors la pile doit être marquée comme étant occupée. Cette contrainte garantit qu'une pile n'est pas utilisée pour placer des blocs si elle est déjà occupée.

1.2.1 Objectif

L'objectif principal de **BWBasicConstraintsBuilder** est de définir un ensemble de contraintes qui gère les relations entre les blocs et les piles dans le monde des blocs. Les contraintes sont définies pour garantir que :

- Les blocs ne peuvent pas occuper la même position en même temps.
- Si un bloc est placé sur un autre, le bloc sous-jacent doit être fixé.
- Si un bloc est placé sur une pile, la pile doit être marquée comme occupée.

Ces contraintes permettent de définir les conditions sous lesquelles les actions peuvent être effectuées dans le monde des blocs.

1.2.2 Méthodes principales

Résumé des méthodes principales de la classe :

- **BWBasicConstraintsBuilder(int nbBlocks, int nbStacks)** : Constructeur qui initialise le nombre de blocs et de piles, puis génère les variables nécessaires et crée les contraintes de base.
- **createConstraints()** : Méthode qui génère toutes les contraintes nécessaires pour le monde donné. Elle définit des contraintes de différence entre les blocs et des implications pour assurer que les blocs et les piles respectent les règles de fixation et de liberté.
- **getConstraints()** : Retourne l'ensemble des contraintes créées.

1.3 BWRegularConstraintsBuilder

La classe **BWRegularConstraintsBuilder** est responsable de la création des contraintes de régularité dans le monde des blocs. Ces contraintes imposent des règles sur la manière dont les blocs peuvent être positionnés les uns par rapport aux autres au sein d'un ensemble défini de piles.

1.3.1 Objectif

L'objectif principal de **BWRegularConstraintsBuilder** est de définir des configurations valides pour les blocs. En particulier, elle crée des contraintes qui :

- Imposent que lorsqu'un bloc **a** est posé sur un bloc **b**, la position de **b** devient restreinte. En effet, **b** ne peut être posé que sur un autre bloc **c** tel que la différence de position entre **b** et **c** soit la même que celle entre **a** et **b** ou directement sur l'une des pile. Cela garantit une certaine régularité dans la disposition des blocs.
- Assurent une régularité dans la disposition des blocs liés (**se trouvant sur la meme pile**).

1.3.2 Méthodes principales

Résumé des méthodes principales de la classe :

- **BWRegularConstraintsBuilder(int nbBlocks, int nbStacks)** : Constructeur qui initialise le nombre de blocs et de piles, puis appelle **createConstraints()** pour générer les contraintes associées.
- **createConstraints()** : Méthode qui crée les contraintes de régularité. Ces contraintes gèrent la manière dont les blocs peuvent être empilés les uns sur les autres dans une meme pile en fonction des différences de position.
- **getConstraints()** : Retourne l'ensemble des contraintes de régularité créées.

1.4 BWCroissanceConstraintsBuilder

La classe **BWCroissanceConstraintsBuilder** est responsable de la création de contraintes qui imposent une règle de **croissance ordonnée** entre les blocs dans le monde des blocs. Cette règle stipule que chaque bloc doit être placé sur un autre bloc ayant un identifiant plus petit ou directement sur la pile, cela nous garantis une structure d'empilement croissante des blocs.

1.4.1 Objectif

Son objectif est de contraindre chaque bloc à être uniquement placé sur un bloc ayant un indice plus petit, ce qui assure que l'empilement suit une logique croissante.

1.4.2 Fonctionnement

La méthode clé qui applique cette contrainte est `createConstraints()`. Son fonctionnement :

1. Pour chaque variable représentant un bloc (**on**), on récupère le domaine de cette variable.
2. Ensuite, pour appliquer la contrainte de croissance, la méthode parcourt tous les indices de blocs et exclut ceux qui sont plus grands ou égaux à l'indice du bloc actuel.
3. En d'autres termes, pour chaque bloc, on **force** son domaine à n'accepter que des indices de blocs plus petits. Cela est réalisé en retirant du domaine de la variable les valeurs correspondant à des blocs ayant un indice supérieur ou égal à celui du bloc actuel.

1.5 BWActionsBuilder

La classe **BWActionsBuilder** est responsable de la création d'un ensemble d'actions de base qui peuvent être réalisées dans un *monde des blocs* spécifique. Ces actions permettent de déplacer des blocs entre les piles et entre les blocs eux-mêmes.

1.5.1 Objectif

L'objectif principal de cette classe est de générer toutes les actions possibles pour manipuler les blocs dans un *monde* donné, en prenant en compte le nombre de blocs et de piles. Les actions sont définies en fonction des **préconditions** et des **effets**, qui dépendent de l'état actuel des blocs et des piles dans le *monde des blocs*.

1.5.2 Création des Actions

La méthode `createBasicActions` crée toutes les actions possibles en tenant compte des différentes configurations des blocs et des piles. Les actions suivantes sont créées :

- **Déplacer un bloc d'un bloc à un autre bloc** : Cette action implique de déplacer un bloc placé sur un autre bloc vers une nouvelle position sur un autre bloc. Les préconditions incluent que les deux blocs ne soient pas fixes et que le bloc de destination ne soit pas occupé.
- **Déplacer un bloc d'un bloc vers une pile** : Dans cette action, un bloc est déplacé d'un autre bloc vers une pile libre. La pile doit être libre et le bloc actuel ne doit pas être fixé.
- **Déplacer un bloc d'une pile vers un bloc** : Cette action déplace un bloc d'une pile vers un autre bloc. La pile doit être libre et le bloc de destination doit pouvoir recevoir le bloc sans être fixé.
- **Déplacer un bloc d'une pile vers une autre pile** : Dans cette action, un bloc est déplacé d'une pile vers une autre pile. La pile de destination doit être libre, et la pile source doit être également libre après le déplacement.

1.6 Classe DataminingVariableBuilder

La classe **DataminingVariableBuilder** est responsable de la création et de la gestion des variables booléennes qui représentent l'état du *monde des blocs*. Elle génère des variables pour des relations telles que “sur”, “surTable”, “fixé” et “libre”, et fournit des correspondances entre ces variables et leurs descriptions sous forme de chaînes de caractères pour en faciliter l'utilisation.

1.6.1 Objectif

Le rôle principal de cette classe est de générer un ensemble de variables logiques qui permettent de modéliser l'état du *monde des blocs*. Ces variables servent à exprimer des relations entre les blocs et les piles, telles que la position d'un bloc par rapport à un autre, si un bloc est fixé ou si une pile est libre.

1.6.2 Structure des Variables

La classe crée les variables suivantes pour chaque bloc et pile dans le modèle :

Nom de la variable	Description
<code>fixed_X</code>	Ces variables définissent le fait qu'un bloc est fixé ou non. Un bloc est considéré comme fixé si il est en dessous d'un autre bloc.
<code>on_X_Y</code>	Ces variables montrent le fait qu'un bloc X est posé sur un autre bloc Y . Instancié à Vrai, ça signifie que le bloc X est posé sur le bloc Y .
<code>onTable_X_Y</code>	Ces variables sont utilisées pour indiquer qu'un bloc est posé sur une pile spécifique.
<code>free_X</code>	Ces variables donne l'information selon laquelle une pile est libre ou déjà occupé par un bloc.

TABLE 1.1 – Structure des variables créées pour chaque bloc et pile dans le modèle.

1.6.3 Création des Variables

La méthode `buildVariables()` est responsable de la création de ces variables. Pour chaque bloc, elle crée des variables pour indiquer si le bloc est fixé, sur un autre bloc ou sur une pile spécifique. Elle crée également des variables pour chaque pile afin de déterminer si elle est libre ou non.

Les étapes de création des variables sont les suivantes :

- **Création des variables “fixed”** : Pour chaque bloc i , une variable `fixed_i` est créée pour indiquer si le bloc est fixé.
- **Création des variables “on”** : Pour chaque paire de blocs (i, j) où $i \neq j$, une variable `on_i_j` est créée pour indiquer que le bloc i est posé sur le bloc j .
- **Création des variables “onTable”** : Pour chaque bloc i et chaque pile k , une variable `onTable_i_k` est créée pour indiquer que le bloc i est posé sur la pile k .
- **Création des variables “free”** : Pour chaque pile k , une variable `free_k` est créée pour indiquer que la pile est libre.

1.6.4 Obtention de l'Instance de l'État

La méthode `getInstance(List<List<Integer>> state)` prend en entrée une liste de piles, chaque pile étant une liste d'ID de blocs. Elle génère à la fin, un ensemble de variables (un sous ensemble des variables crée par la méthode `buildVariables()`) dont la valeur est Vrai dans le monde représenté par la liste reçue en argument.

1.7 Les différents types d'heuristiques

1.7.1 Heuristique des Blocs Mal Placés

L'heuristique `MisplacedBlock` estime le coût d'un état en fonction du nombre de blocs mal placés par rapport à un état objectif donné. Cette heuristique permet de guider les algorithmes de recherche en fournissant une estimation du nombre de blocs qui ne sont pas dans leurs positions correctes par rapport à l'objectif.

- **But de l'heuristique** : Cette heuristique aide à évaluer la distance entre l'état actuel du monde des blocs et un objectif spécifique en comptabilisant les blocs qui ne sont pas correctement positionnés.
- **Fonctionnement** : La fonction `estimate` parcourt les variables représentant les positions des blocs dans l'état cible (objectif) et compare leur valeur avec celle de l'état courant. Si une valeur diffère, cela signifie que le bloc est mal placé, et l'heuristique augmente le compteur des blocs mal placés.

1.8 Heuristique de Similarité Cosinus

L'heuristique de similarité cosinus est utilisée pour estimer la proximité entre l'état actuel d'un système et un état cible ou objectif dans un problème de planification. L'idée sous-jacente est que plus l'état actuel est similaire à l'état cible, plus la valeur de la similarité est élevée. Cette heuristique repose sur la mesure de l'angle entre deux vecteurs représentant les états, en utilisant la formule de la similarité cosinus. Elle est particulièrement utile pour guider des algorithmes de recherche en donnant une estimation de la "distance" entre deux états.

1.8.1 Explication de l'Heuristique

L'heuristique de similarité cosinus compare deux vecteurs :

- Le vecteur représentant l'état actuel.
- Le vecteur représentant l'état cible ou objectif.

La similarité cosinus mesure l'angle entre ces deux vecteurs. Si les vecteurs sont très similaires (i.e., l'angle entre eux est petit), la valeur de la similarité sera proche de 1, ce qui indique que l'état actuel est proche de l'objectif. À l'inverse, si les vecteurs sont très différents (i.e., l'angle est large), la similarité sera faible, proche de 0 ou même négative.

La formule de la similarité cosinus est la suivante :

$$\text{similarité cosinus}(\vec{A}, \vec{B}) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|}$$

Où :

- $\vec{A}$ et $\vec{B}$ sont des vecteurs représentant respectivement l'état actuel et l'état objectif.
- $\vec{A} \cdot \vec{B}$ est le produit scalaire des deux vecteurs, calculé comme la somme des produits de leurs composantes respectives.
- $\|\vec{A}\|$ et $\|\vec{B}\|$ sont les normes (ou magnitudes) des vecteurs $\vec{A}$ et $\vec{B}$, calculées comme la racine carrée de la somme des carrés de leurs composantes.

1.8.2 Calcul du Produit Scalaire et de la Norme

Le produit scalaire $\vec{A} \cdot \vec{B}$ est défini par :

$$\vec{A} \cdot \vec{B} = \sum_{i=1}^n A_i \times B_i$$

Où A_i et B_i sont les composantes des vecteurs $\vec{A}$ et $\vec{B}$. Pour des variables booléennes, si $A_i = B_i$, alors $A_i \times B_i = 1$, sinon $A_i \times B_i = 0$.

La norme d'un vecteur $\vec{A}$ est donnée par :

$$\|\vec{A}\| = \sqrt{\sum_{i=1}^n A_i^2}$$

1.8.3 Estimation de la Similarité Cosinus

La similarité cosinus entre l'état actuel $\vec{A}$ et l'état objectif $\vec{B}$ est calculée comme suit :

$$\text{similarité cosinus} = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|}$$

Cette mesure varie entre -1 et 1 :

- 1 indique une correspondance parfaite entre les deux états.
- 0 indique qu'il n'y a pas de correspondance.
- -1 indique que les états sont complètement opposés.

Plus la similarité est élevée, plus l'état actuel est proche de l'objectif. Cette heuristique est donc utilisée pour évaluer la "distance" entre l'état courant et l'objectif, et ainsi guider les algorithmes de recherche vers la solution.

1.8.4 Exemple d'Application

Soit $\vec{A} = [1, 0, 1, 1]$ et $\vec{B} = [1, 1, 0, 1]$.

1. ****Produit scalaire**** :

$$\vec{A} \cdot \vec{B} = (1 \times 1) + (0 \times 1) + (1 \times 0) + (1 \times 1) = 2$$

2. ****Normes**** :

$$\|\vec{A}\| = \sqrt{1^2 + 0^2 + 1^2 + 1^2} = \sqrt{3}, \quad \|\vec{B}\| = \sqrt{1^2 + 1^2 + 0^2 + 1^2} = \sqrt{3}$$

3. ****Similarité cosinus**** :

$$\text{similarité cosinus} = \frac{2}{\sqrt{3} \times \sqrt{3}} = \frac{2}{3} \approx 0.666$$

Cet exemple montre que la similarité entre les deux états est de 0.666, ce qui indique que les deux états sont relativement proches l'un de l'autre, mais il existe des différences.

Chapitre 2

Expérimentation

2.1 Compilation et exécution

Cette session est une reprise des commandes de compilation et d'exécution décrites dans le README.md du projet. Se placer dans le dossier src et lancer les commandes suivantes :

2.1.1 Compilation

— *javac -d ../build -cp ../lib/bwgenerator.jar ../lib/blocksworld.jar planning/Utils/*.java planning/*.java modelling/*.java datamining/*.java cp/*.java blocksworld/*.java blocksworld/Utils/*.java blocksworld/*.java blocksworld/demo/*.java*

2.1.2 Exécution des démos

Démos	Commande
Vérification 1	java -cp ../build ../lib/blocksworld.jar blocksworld.demos.VerificationSatisfactionConstraint1
Vérification 2	java -cp ../build ../lib/blocksworld.jar blocksworld.demos.VerificationSatisfactionConstraint2
Configuration 1	java -cp ../build ../lib/blocksworld.jar blocksworld.demos.CreationConfiguration1
Configuration 2	java -cp ../build ../lib/blocksworld.jar blocksworld.demos.CreationConfiguration2
Configuration 3	java -cp ../build ../lib/blocksworld.jar blocksworld.demos.CreationConfiguration3
Planning1	java -cp ../build ../lib/blocksworld.jar blocksworld.demos.Planning1
Planning2	java -cp ../build ../lib/blocksworld.jar blocksworld.demos.Planning2
Datamining	java -cp ../build ../lib/blocksworld.jar blocksworld.demos.Datamining

TABLE 2.1 – Commande d'exécution des démos

2.2 Démonstration

2.2.1 Vérification de satisfaction de contraintes

Dans ce démo, nous avons crée deux mondes des blocs et des contraintes pour chacun d'eux. Par la suite, nous avons essayé de vérifier si toutes les contraintes sont satisfaites ou non. La première contrainte non satisfaite sur l'image 2.1 est celle imposant le bloc 6 à se poser sur un bloc de numéro inférieur. La deuxième est celle imposant une configuration régulière. Plus explicitement, il s'agit de la contrainte qui impose au bloc 7 de ce posé sur une pile si le bloc 6 est posé sur ce dernier dans un monde qui se doit d'être régulier.

```
On a un monde représenté par [[1,2],[],[0]]
On crée tous les types de contraintes : Basiques,Regulières,Croissantes
Toutes les contraintes sont satisfaites dans ce monde ✓

On a un monde représenté par [[0,1,2],[3,7,6],[4,5]]
On crée tous les types de contraintes :Basiques,Regulières,Croissantes"
Contrainte d'implication portant sur Variable : 6 Domaine : [-1, 0, -2, 1, -3, 2, 3, 4, 5, 7]. S1 = [-1, 0, -2, 1, 3, 2, 3, 4, 5, 7] et Variable : 6 Domaine : [-1, 0, -2, 1, -3, 2, 3, 4, 5, 7]. S2 = [-1, 0, -2, 1, -3, 2, 3, 4, 5] n'est pas satisfaite ✗
Contrainte d'implication portant sur Variable : 6 Domaine : [-1, 0, -2, 1, -3, 2, 3, 4, 5, 7]. S1 = [7] et Variable : 7 Domaine : [-1, 0, -2, 1, -3, 2, 3, 4, 5, 6]. S2 = [-1, -2, -3] n'est pas satisfaite ✗
```

FIGURE 2.1 – Vue sur les résultats de vérification de satisfaction de contrainte

2.2.2 Création de monde

Dans les démos, nous avons essayé de générer un monde composée de 6 blocs et 3 piles avec les contraintes basiques et régulières. Avec les trois solveurs, nous avons comme mondes, ceux affiché sur les images suivantes. Dans la console, nous avons une vue montrant les performances de chaque solveur.

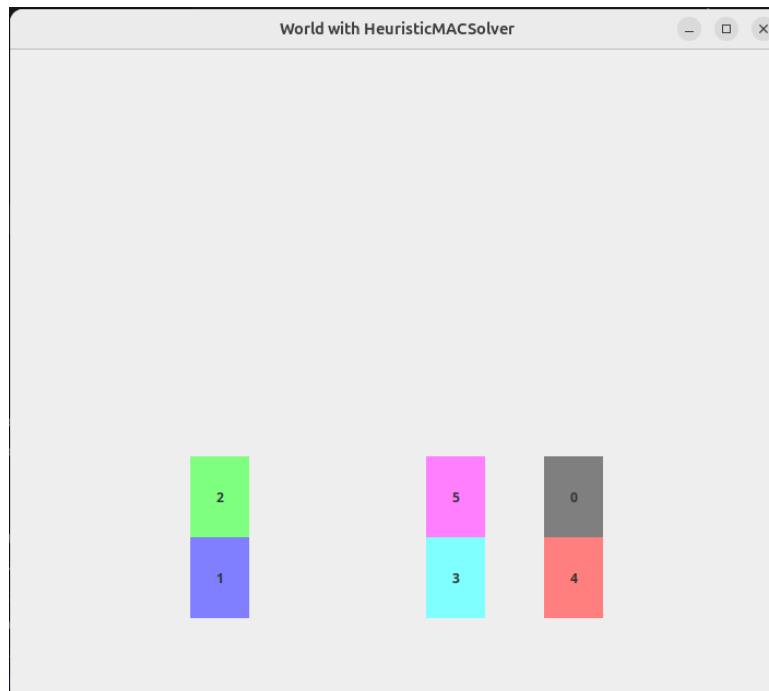


FIGURE 2.2 – Monde générer par HeuristicMACSolver

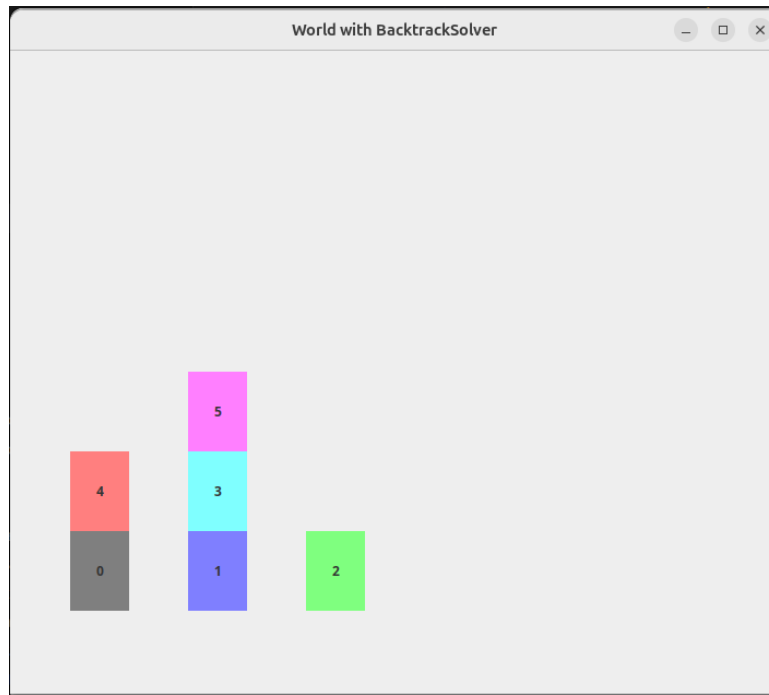


FIGURE 2.3 – Monde générer par BacktrackSolver

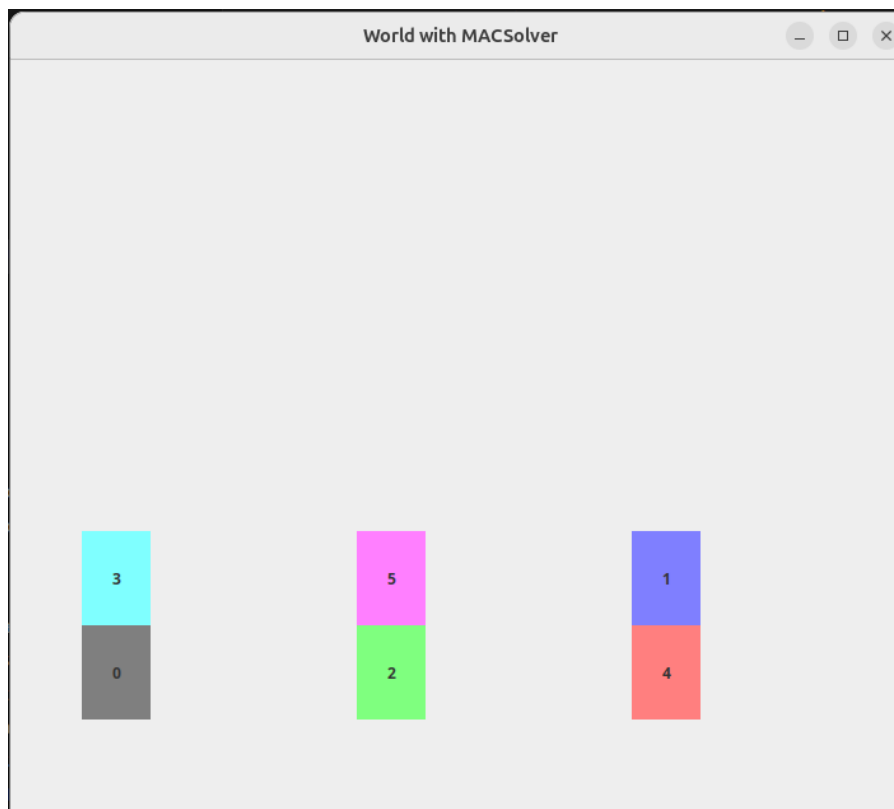


FIGURE 2.4 – Monde générer par MACSolver

2.2.3 Planning

La figure 2.6 montre les résultats de la démo définie dans la classe Planner1. Nous y retrouvons les performances de chaque algorithme de planification et le plan trouvé par A* avec l'heuristique de blocs mal placés.


```

On cherche une configuration composée de 6 blocks et 3 piles
Contraintes : basiques et régulière

Résultat BacktrackSolver
Temps d'exécution : 25 ms

Résultat MACSolver
Temps d'exécution : 495 ms

Résultat HeuristicMACSolver
Temps d'exécution : 148 ms

```

FIGURE 2.5 – Vue sur les performances des solveurs

```

Init : [[0,2,1],[4,3],[]]
Goal : [[4,2],[1],[3,0]]

Résultat DFSPlanner
Temps d'exécution : 86ms
Nombre d'actions dans le plan : 1082
Nombre de noeuds explorés : 1241

Résultat BFSPlanner
Temps d'exécution : 101ms
Nombre d'actions dans le plan : 8
Nombre de noeuds explorés : 915

Résultat DijkstraPlanner
Temps d'exécution : 212ms
Nombre d'actions dans le plan : 8
Nombre de noeuds explorés : 2520

Résultat AStarPlanner avec l'heuristique de similarité
Temps d'exécution : 116ms
Nombre d'actions dans le plan : 8
Nombre de noeuds explorés : 1889

Résultat AStarPlanner avec l'heuristique bloc mal placé
Temps d'exécution : 3ms
Nombre d'actions dans le plan : 8
Nombre de noeuds explorés : 70

Simulation of plan: done.
Ci dessous le plan exécuté

[Préconditions : {BooleanVariable : 3=false, Variable : 3 Domaine : [-1, 0, -2, 1, -3, 2, 4].=-4, BooleanVariable : -3=true}
Effets : {BooleanVariable : 4=false, Variable : 3 Domaine : [-1, 0, -2, 1, -3, 2, 4].=-3, BooleanVariable : -3=false}
, Préconditions : {Variable : 4 Domaine : [-1, 0, -2, 1, -3, 2, 3].=-2, BooleanVariable : 3=false, BooleanVariable : 4=false}
Effets : {BooleanVariable : -2=true, Variable : 4 Domaine : [-1, 0, -2, 1, -3, 2, 3].=3, BooleanVariable : 3=true}
, Préconditions : {BooleanVariable : -2=true, BooleanVariable : 1=false, Variable : 1 Domaine : [-1, 0, -2, -3, 2, 3, 4].=-2}
Effets : {BooleanVariable : -2=false, BooleanVariable : 2=false, Variable : 1 Domaine : [-1, 0, -2, -3, 2, 3, 4].=-2}
, Préconditions : {BooleanVariable : 1=false, BooleanVariable : 2=false, Variable : 2 Domaine : [-1, 0, -2, 1, -3, 3, 4].=-0}
Effets : {BooleanVariable : 1=true, BooleanVariable : 0=false, Variable : 2 Domaine : [-1, 0, -2, 1, -3, 3, 4].=-1}
, Préconditions : {BooleanVariable : 0=false, BooleanVariable : 2=false, Variable : 0 Domaine : [-1, -2, 1, -3, 2, 3, 4].=-1}
Effets : {BooleanVariable : -1=true, BooleanVariable : 2=true, Variable : 0 Domaine : [-1, -2, 1, -3, 2, 3, 4].=-2}
, Préconditions : {BooleanVariable : -1=true, Variable : 4 Domaine : [-1, 0, -2, 1, -3, 2, 3].=-3, BooleanVariable : 4=false}
Effets : {BooleanVariable : -1=false, Variable : 4 Domaine : [-1, 0, -2, 1, -3, 2, 3].=-1, BooleanVariable : 3=false}
, Préconditions : {BooleanVariable : 0=false, BooleanVariable : 3=false, Variable : 0 Domaine : [-1, -2, 1, -3, 2, 3, 4].=-2}
Effets : {BooleanVariable : 3=true, BooleanVariable : 2=false, Variable : 0 Domaine : [-1, -2, 1, -3, 2, 3, 4].=-3}
, Préconditions : {BooleanVariable : 2=false, BooleanVariable : 4=false, Variable : 2 Domaine : [-1, 0, -2, 1, -3, 3, 4].=-1}
Effets : {BooleanVariable : 1=false, BooleanVariable : 4=true, Variable : 2 Domaine : [-1, 0, -2, 1, -3, 3, 4].=-4}
]

```

FIGURE 2.6 – Vue sur les performances des classes de planification

2.2.4 Datamining

Dans la démo de datamining, nous avons généré 15 mondes composés de 10 blocs et 3 piles. Avec la méthode getInstance nous avons pu obtenir les variables instanciées à Vrai pour chaque monde. Les résultats de l'extraction de donnée réalisée sur la base de données créée sont affichés dans les figures 2.7 et 2.8.

```
DATAMINING
15 Mondes de 10 blocs et 3 piles générés aléatoirement
MOTIF DE FREQUENCE += 2/3
fixed_0
Frequency = 0.6666667

fixed_7
Frequency = 0.73333335

fixed_1
fixed_5
Frequency = 0.6666667

fixed_5
Frequency = 0.8666667

fixed_1
Frequency = 0.6666667

fixed_8
Frequency = 0.73333335

fixed_2
Frequency = 0.6666667

fixed_4
Frequency = 0.6666667

fixed_2
fixed_5
Frequency = 0.6666667

fixed_6
Frequency = 0.73333335

fixed_3
Frequency = 0.73333335
```

FIGURE 2.7 – Vue sur les performances des classes d'extraction de connaissances

```
REGLE D'ASSOCIATION DE FREQUENCE += 2/3 ET DE CONFIANCE += 95/100
Premise:
fixed_2
Conclusion:
fixed_5
Frequency = 0.666667
Confidence = 1.0

Premise:
fixed_1
Conclusion:
fixed_5
Frequency = 0.666667
Confidence = 1.0
```

FIGURE 2.8 – Vue sur les performances des classes d'extraction de connaissances

Conclusion

En conclusion, ce projet a été une opportunité pour nous de mettre en pratique les concepts d'aide à la prise de décision et d'intelligence artificielle que nous avons étudiés tout au long de ce semestre. Travailler sur le problème du monde des blocs nous a permis de mieux comprendre et d'appliquer des notions telles que la **satisfaction de contraintes** et la **planification** dans un cadre concret. Ce travail a également renforcé notre maîtrise des outils vus en cours, tout en nous poussant à réfléchir de manière autonome et créative pour développer des solutions adaptées. Il s'agit donc d'un exercice enrichissant, à la fois intellectuellement et techniquement, qui nous a permis de relier théorie et pratique de manière concrète .

Bibliographie

- [1] Outil utilisé pour le rapport *Latex* .
- [2] *Trello*, <https://trello.com/fr>
- [3] *Métriques de similarité*, <https://milvus.io/docs/fr/metric.md?tab=floating>,
- [4] *Similarité cosinus*, https://fr.wikipedia.org/wiki/Similarit%C3%A9_cosinus,
- [5] *Documentation java*, <https://docs.oracle.com/en/java/javase/11/docs/api/>.